



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

Subject : Artificial Intelligence (SECJ3553)

Semester : Semester 1/20252026

Section : Section 03

Theme : Smart Education (SimuLearn)

Task : Assignment 2

Lecturer : Dr. Norsham Idris

Prepared by:

No.	Name	Matric Number
1	Lee Yin Shen	A23CS0236
2	Joanne Ching Yin Xuan	A23CS0227
3	Muhammad Afiq Danish bin Mohd Hazni	A23CS0118
4	Gui Kah Sin	A23CS0080

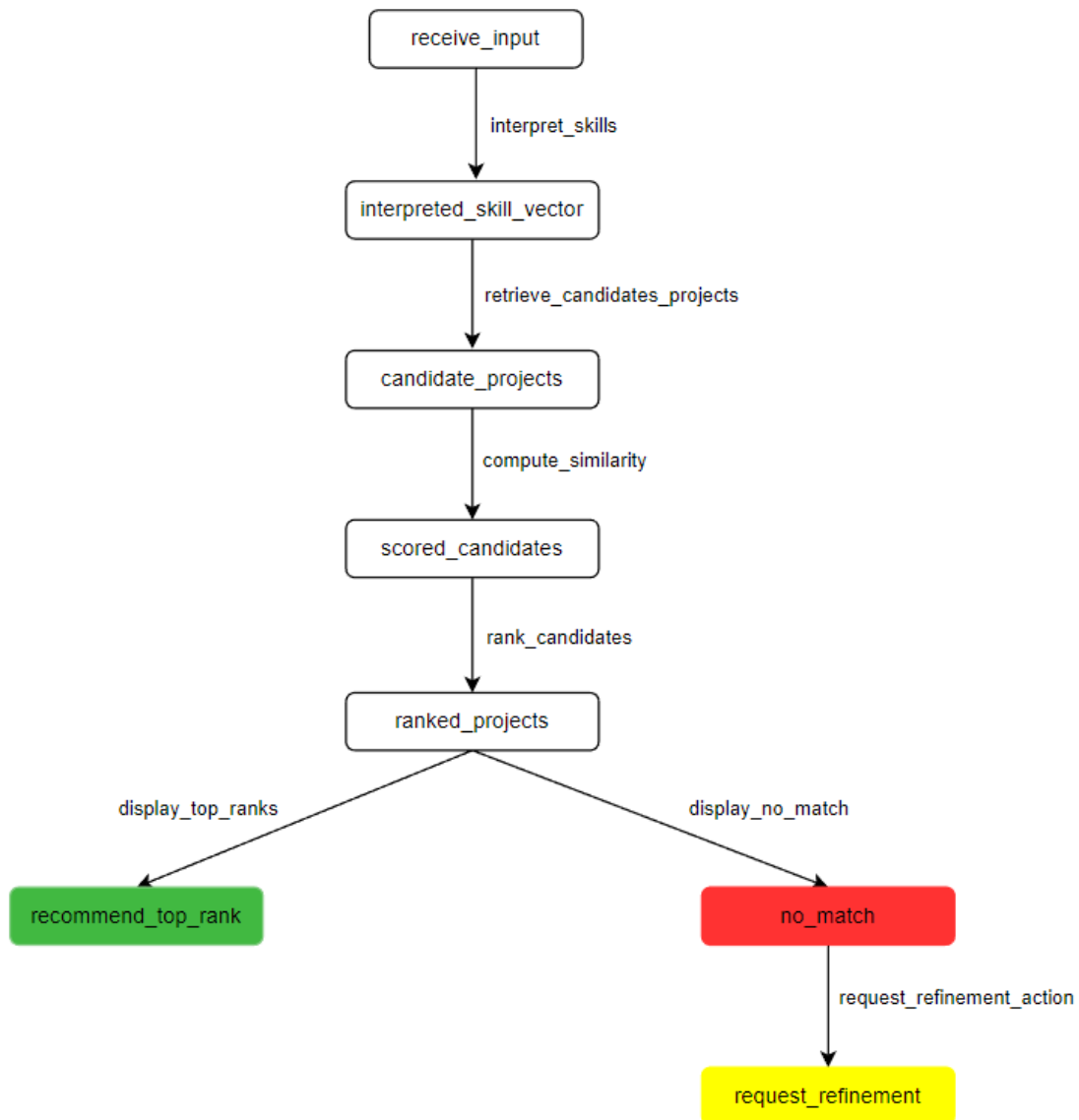
Table of Content

State Space Search: SimuLearn.....	3
1.0 Intelligent Project Matching.....	3
1.1 State Space Graph.....	3
1.2 States and Actions Explanation.....	3
1.3 Problem Formulation and Explanation.....	5
1.4 Solution.....	6
2.0 Guided Task Breakdown.....	7
2.1 State Space Graph.....	7
2.3 Problem Formulation and Explanation.....	8
2.4 Solution.....	9
3.0 Career Suggestion System.....	10
3.1 State Space Graph.....	10
3.2 States and Actions Explanation.....	10
3.3 Problem Formulation and Explanation.....	11
3.4 Solution.....	13
4.0 How Problem Formulation Supports the Proposed KR.....	14

State Space Search: SimuLearn

1.0 Intelligent Project Matching

1.1 State Space Graph



1.2 States and Actions Explanation

States	a. receive_input Initial state where the student enters text input such as topic learned, skills or syllabus keywords. The system captures the raw input before any processing begins.
--------	--

	b. interpreted_skill_vector The system converts the student's raw input into a structured list of recognized skills including the keyword extraction to interpret the student's abilities. c. candidate_projects The system retrieves all relevant simulated projects that share at least partial skill overlap with the student skills. d. scored_candidates The state where each retrieved projects are evaluated. The system computes a match score based on similarity between student skills and project requirements. e. ranked_projects The system sorts projects from highest to lowest match score. The ranked list is prepared to determine the best possible recommendation for the student. f. recommend_top_rank The system selects the highest-ranked project and recommends it to the student, showing match score, covered skills, missing skills and suggested starting tasks. g. no_match The state where it is triggered when no project meets the minimum matching threshold indicates that the similarity is too low or no overlapping skills. The system cannot provide a suitable recommendation. h. request_refinement The state where the system prompts the student to refine, re-enter or expand their input.
Actions	a. interpret_skills The system processes the student's input. b. retrieve_candidates_projects The system queries and retrieves all projects that require the related skills. c. compute_similarity The system calculates similarity or match scores for each candidate project. d. rank_candidates The system sorts all candidate projects from the best match to the weakest match. e. display_top_ranks The system recommends the best-matched project to the student. f. display_no_match The system displays "no match" results when no project is suitable. g. request_refinement_action The system asks the student to improve their input. This action returns the system to the initial states for a new matching cycle.

1.3 Problem Formulation and Explanation

Critical	Details	Explanation
Initial State	receive_input	The student submits their desired learning topics or skills. No matching has occurred yet and no interpretation has been performed.
Action	Let $S(x)$ = set of state transitions S(receive_input) = { interpret_skills → interpreted_skill_vector } S(interpreted_skill_vector) = { retrieve_candidate_projects → candidate_projects } S(candidate_projects) = { compute_similarity → scored_candidates } S(scored_candidates) = { rank_candidates → ranked_projects } S(ranked_projects) = { display_top_ranks → recommend_top_rank, display_no_match → no_match } S(no_match) = { request_refinement_action → request_refinement }	These transitions represent the decision-making flow of the Intelligent Project Matching system. Using KR1, the system retrieves projects based on skill overlap. Using KR2, the system interprets the student's input. The system transitions to different states depending on whether matching is successful or not. This simulates a realistic matching workflow where recommendations are only given when alignment is strong enough.
Goal	recommend_top_rank	The goal state is achieved when the system successfully identifies one or more projects that best match the student's skills and presents them as recommendations.
Path Cost	Cost on the route: receive_input → interpreted_skill_vector → candidate_projects → scored_candidates → ranked_projects → recommend_top_rank	Path cost represents the number of processing steps required before producing the recommendation. If a match is strong, the system reaches the goal quickly. If the similarity score is low, the system enters failure states (no_match, request_refinement) the total number of steps increasing.

1.4 Solution

Solution 1: System successfully produce the recommendations

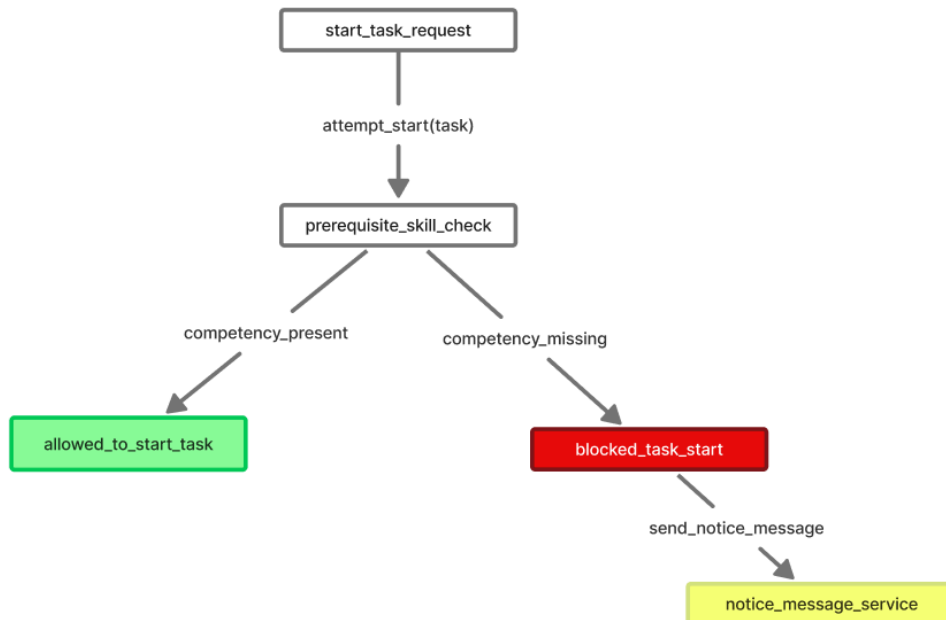
receive_input → *interpreted_skills* → interpreted_skill_vector →
retrieve_candidates_projects → candidate_projects → *compute_similarity* →
scored_candidates → *rank_candidates* → ranked_projects → *display_top_ranks* →
recommend_top_rank

Solution 2: System fail to produce the recommendations

receive_input → *interpreted_skills* → interpreted_skill_vector →
retrieve_candidates_projects → candidate_projects → *compute_similarity* →
scored_candidates → *rank_candidates* → ranked_projects → *display_no_match* →
no_match → *request_refinement_action* → request_refinement

2.0 Guided Task Breakdown

2.1 State Space Graph



2.2 States and Actions Explanation

States	a. start_task_request Initial state where the student chooses a task to begin a task in the SimuLearn platform. b. prerequisite_skill_check The system verifies whether the student possesses the required prerequisite competency for the selected task. c. allowed_to_start_task The state indicates that the student has the necessary prerequisite competency and is permitted to begin the task. d. blocked_task_start The state where the system identifies that the student is missing required competency and prevents the task from starting. e. notice_message_service A notification state where the system sends a message to the student explaining that they must complete the prerequisite before proceeding.
Actions	a. attempt_start(task)

	Student attempts to initiate a selected task.. b. competency_present Condition indicating that the student has the required prerequisite. c. competency_missing Condition indicating the student does not meet the prerequisite competency. d. send_notice_message The system sends a notice to inform the student that they must complete the missing prerequisite.
--	---

2.3 Problem Formulation and Explanation

Critical	Details	Explanation
Initial State	start_task_request	The student has not completed any prerequisites and initiates a request to begin a chosen task.
Action	Let $S(x)$ = set of action–state transitions. S(start_task_request) = { attempt_start(task) → prerequisite_skill_check } S(prerequisite_skill_check) = { competency_present → allowed_to_start_task, competency_missing → blocked_task_start } S(blocked_task_start) = { send_notice_message → notice_message_service }	This formulation ensures the SimuLearn AI enforces skill-based prerequisites. Under KR3, the system blocks any task attempt if the required competency is missing. Under KR4, once the competency is obtained, the student is allowed to begin the task. This approach simulates real-world project workflow where prerequisite knowledge must be met before progressing..
Goal	allowed_to_start_task	The goal state is reached when the student has fulfilled the prerequisite competency and is permitted to begin the selected task.
Path Cost	Cost on the route: start_task_request, prerequisite_skill_check, blocked_task_start, notice_message_service,	Path cost represents the number of transitions required before the task becomes accessible. A student with the required competency reaches the

		goal quickly. If competency is missing, additional steps are taken (notice message, prerequisite completion) before reaching the goal.
--	--	--

2.4 Solution

Solution 1: Student meets prerequisite competency

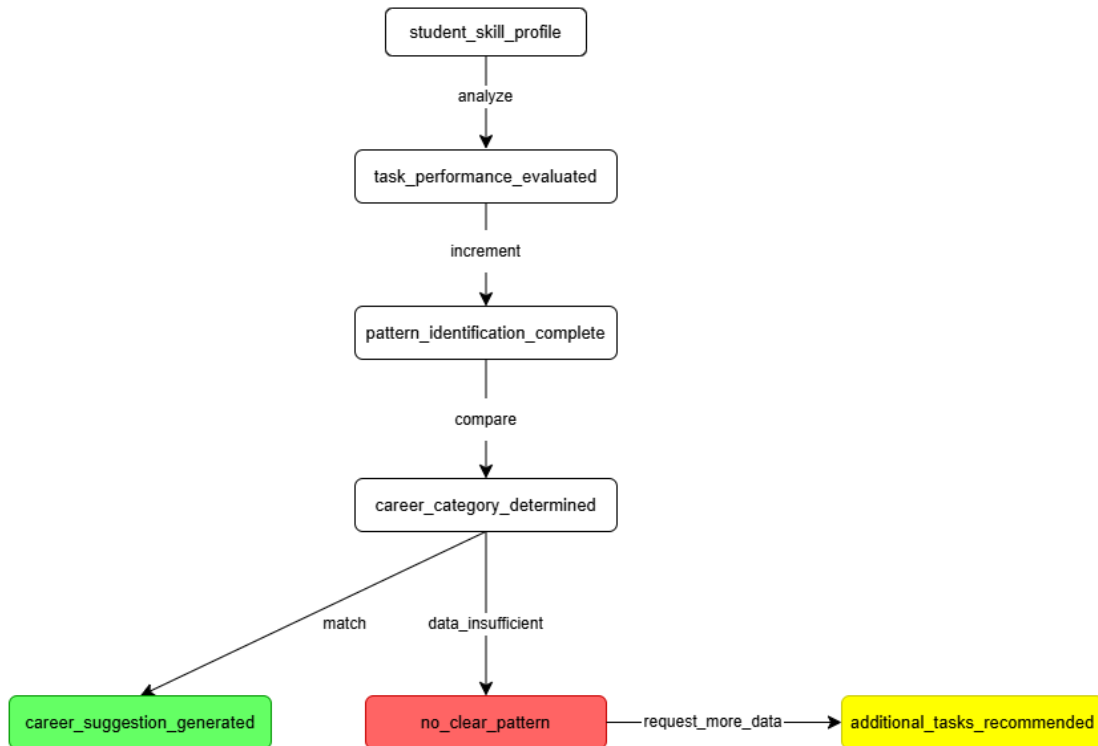
start_task_request → *attempt_start(task)* → prerequisite_skill_check → *competency_present*
→ allowed_to_start_task

Solution 2: Student does NOT meet prerequisite competency

start_task_request → *attempt_start(task)* → prerequisite_skill_check → *competency_missing*
→ blocked_task_start → *send_notice_message* → notice_message_service

3.0 Career Suggestion System

3.1 State Space Graph



3.2 States and Actions Explanation

States	a. student_skill_profile Initial state where the system has retrieved all completed tasks and category counters for the student. b. task_performance_evaluated The system analyses all completed tasks by professional domain (database, UI, Backend, etc.). c. pattern_identification_complete The state increases the student's profile counters for all relevant categories, establishing preference patterns. d. career_category_determined The state where the system compares category counters and identifies the domain with the strongest performance or preference pattern. e. career_suggestion_generated Goal state. The AI has formulated a definitive career suggestion based on the identified pattern.
--------	--

	f. no_clear_pattern State indicating the system could not generate a confident prediction due to insufficient data or equally distributed counters. g. additional_tasks_recommended Final state for the failure path where the system requests the student to complete more additional tasks for producing a confident career prediction.
Actions	a. analyze Action to retrieve student performance records and move to evaluation phase. b. evaluate Action to process raw data and categorize tasks. c. increment Action to increase the profile counter based on task category. d. compare The system compares category counters against thresholds. e. match Action to match the strongest skill pattern to a potential career path. f. data_insufficient Condition taken when the pattern comparison is inconclusive. g. request_more_data Action to generate the message asking the student to continue engaging more tasks.

3.3 Problem Formulation and Explanation

Critical	Details	Explanation
Initial State	student_skill_profile	The process begins when the system is prompted to generate a suggestion, starting by retrieving the student's historical performance data.
Action	Let $S(x)$ = set of action–state transitions. $S(\text{student_skill_profile}) = \{ \text{analyze} \rightarrow \text{task_performance_evaluated} \}$	The analyze action occurs when the system queries the student's records (completed tasks, assigned categories, and current profile counters for each categories)

	S(task_performance_evaluated) = { increment → pattern_identification_complete }	The increment action is the execution of KR5. This action subsumes the evaluate action, where the system first evaluates the task's category (Action b) and then executes the counter increase (Action c).
	S(pattern_identification_complete) = { compare → career_category_determined }	The compare action is the start of KR6 where the system compares all category counters against the predefined Threshold for confidence.
	S(career_category_determined) = {{match → career_suggestion_generated}(Success Path), {data_insufficient → no_clear_pattern}(Failure Path) }	The match action occurs by mapping the dominant skill category to a specific career title, but if only the prediction is confident. The condition data_insufficient is a flag raised by comparison logic when the confidence threshold is not reached.
	S(no_clear_pattern) = { request_more_data → additional_tasks_recommended}	The request_more_data action will compile the message to the student and request the student to complete more tasks for prediction.
Goal	career_suggestion_generated	The goal state is reached when the final career suggestion is presented to the student.
Path Cost	Total inference steps Cost on the route: student_skill_profile, task_performance_evaluated, pattern_identification_complete, career_category_determined, career_suggestion_generated	The path cost is the total number of actions taken (Cost = 1 per action). The Path Cost represents computational effort required to reach the goal.

3.4 Solution

Solution 1 (Success and Optimal Path):

student_skill_profile → *analyze* → task_performance_evaluated → *increment* →
pattern_identification_complete → *compare* → career_category_determined → *match* →
career_suggestion_generated

4.0 How Problem Formulation Supports the Proposed KR

The problem formulation of the **Intelligent Project Matching System** supports **KR1** and **KR2** by structuring the search into a filtering process. The state space diagram first interprets the student's original input as a structured skill markup, which is the actual implementation of **KR2**. Then, search and continue to calculate the similarity and ranking of the project, and apply the logic of **KR1** to find the best match between what students know and what the project needs. By defining the target status as the `recommend_top_rank`, the search ensures that the system actively filters the most relevant items instead of accepting random input, which achieves the goal of personalised recommendation.

For **Guided Task Breakdown System**, the state space diagram show the prerequisite logic defined in **KR3** and **KR4**. The diagram uses the central decision node `prerequisite_skill_check` as the keeper. If the student lacks skills, the path branches into the "blocked" state and executes **KR3**. On the contrary, if the skill exists, the path branches to the "allowed" state and executes **KR4** to unlock the task. This structure ensures that the path to the target state '`allowed_to_start_task`' follows a logical learning pattern and prevents students skip the necessary steps.

Lastly, the **Career Suggestion System** supports **KR5** and **KR6** by modelling the search into a data accumulation process. The diagram uses incremental operations to track students' performance, which is the operation version of the **KR5** pattern identification. Then, the key comparison node executes **KR6** by checking whether these accumulation patterns meet a specific threshold. Only when the evidence is strong enough, the search will branch to the target state '`career_suggested_generated`', otherwise it will turn to a retreat to ensure that the career advice of artificial intelligence is always based on reliable data.